THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Chosen Software**: Git**

Student Name**: Dinesh Kumar**

Registration Number: **24BAI10628**

Date**: 31th March 2026**

1. Introduction

Imagine working on a large software project with dozens of collaborators scattered across the globe. Someone makes a change that breaks everything. You need to find out who made it, when they made it, and exactly what changed — then roll it back without losing everything else that happened in the meantime. Without the right tool, this scenario is a nightmare. With Git, it takes seconds.

This report is a deep audit of Git — one of the most widely used pieces of software in the world, even though most people who benefit from it have no idea it exists. Git is a distributed version control system, released under the GNU General Public License version 2 (GPL v2). It was created by Linus Torvalds in 2005, not as a commercial product, but out of necessity and frustration. That origin story alone tells you everything important about why open source exists.

This report covers the complete lifecycle of Git as an open-source project: why it was built, what its license allows, the philosophical questions its existence raises, how it runs on a Linux system, and how it compares against proprietary alternatives. Each section is followed by my own analysis and reflection — not a summary of what others have written, but what I understood and concluded from reading, installing, and using Git myself.

2. Part A1 — Origin Story: The Problem Git Solved

2.1 The World Before Git

To understand why Git matters, you have to understand how miserable software collaboration was before it existed. In the 1990s and early 2000s, development teams used centralized version control systems — tools like CVS (Concurrent Versions System) and later Subversion (SVN). The concept was simple: there is one central server that holds the authoritative version of the code. Developers check out a copy, make changes, and check those changes back in.

This worked when projects were small and teams sat in the same building. But it had deep structural weaknesses. The central server was a single point of failure. If the server went down, no one could commit code, view history, or do anything version-control-related. If the server's disk died without a backup, the entire history of the project could be lost overnight. And for large, distributed projects — the kind where contributors span continents and time zones — this centralized model was genuinely painful to use at scale.

There were also social and political problems. Who owned the central server? Who had write access? In open-source projects, every contributor who wanted to commit a change had to be given special permission by whoever controlled the repository. This created bottlenecks, political conflicts, and a barrier to casual contribution that contradicted the open spirit of free software development.

2.2 The Linux Kernel's Crisis

Linus Torvalds had been maintaining the Linux kernel — arguably the most important open-source project in the world — since 1991. By 2002, the kernel had grown to a scale where CVS and patch files were simply unworkable. After significant community debate, the Linux kernel team started using a proprietary version control system called BitKeeper.

BitKeeper was good. It was fast, it handled large distributed repositories, and its creator, Larry McVoy, had generously offered it free of charge to the open-source community. But it remained proprietary software — meaning the source code was not available, and the community's use of it was conditional on the goodwill of a single company. For a project founded on the principles of freedom and open collaboration, this dependency felt deeply uncomfortable.

In 2005, that discomfort became a crisis. A developer named Andrew Tridgell began reverseengineering the BitKeeper protocol, apparently to understand how it worked and potentially to write open-source tools that could interoperate with it. BitKeeper's creator, Larry McVoy, saw this as a violation of the spirit of the free license agreement and revoked the open-source community's ability to use BitKeeper without paying.

Linus Torvalds was left without a version control system for the Linux kernel. He could have adopted one of the existing free tools, but he evaluated each of them and found them all fundamentally inadequate for the scale and distributed nature of the kernel project. CVS was too slow. SVN had improved on CVS but was still centralized. Other experimental systems existed but were immature.

So Linus did something that sounds extreme but is actually very typical of the open-source world: he wrote his own.

2.3 The Ten-Day Creation

In April 2005, Linus Torvalds spent roughly ten days writing the initial version of Git from scratch. His design goals were clear from the start, and he stated them publicly on the Linux kernel mailing list. Git had to be fast — fast enough to handle the Linux kernel's enormous history. It had to be distributed — every copy of the repository had to be a complete, self-sufficient mirror, with no single server acting as a central authority. It had to have strong guarantees of data integrity — every file, every commit, every piece of history had to be cryptographically hashed so that any corruption or tampering would be immediately detectable. And it had to support non-linear development — the ability to create branches cheaply and merge them efficiently.

The name 'Git' was a joke, in the British sense of the word — British slang for a foolish or contemptible person. Linus has said, in his characteristic self-deprecating style, that he names all his projects after himself: first Linux, now Git. The joke stuck.

What is remarkable about the origin of Git is not just the speed of its creation but the clarity of its design philosophy. Linus was not building a product for sale. He was not trying to corner a market. He was solving a real problem that he personally faced, and he shared the solution with the world under the GPL — the same license that protects the Linux kernel itself. This is the essence of opensource culture: scratch your own itch, then give others the tool to scratch theirs.

2.4 What Changed After Git

Git was so well-designed that it rapidly overtook every other version control system. By 2010, major projects were migrating from SVN and CVS to Git. The launch of GitHub in 2008 gave Git a social platform that made collaboration even easier — though it is important to understand that GitHub is built on top of Git, not the same thing as Git. Git is the open-source engine; GitHub is a proprietary hosting service that runs that engine in the cloud.

Today, Git is the default version control system for essentially every software project in the world, from solo developers to teams at Google, Microsoft, Apple, and Facebook. Every time someone commits code on GitHub, every time a CI/CD pipeline runs, every time a software package is installed from a repository — Git is almost certainly involved somewhere in the chain. It is, as the rubric for this project

3. Part A2 — License Analysis: What GPL v2 Actually Means

3.1 The Four Freedoms of Free Software

Richard Stallman's Free Software Foundation defines free software through four essential freedoms. Understanding these freedoms is not academic — they directly determine what you can and cannot do with any software, including Git.

- Freedom 0: The freedom to run the program for any purpose, in any way you choose, without restriction.
- Freedom 1: The freedom to study how the program works and modify it to suit your needs. Access to the source code is a prerequisite for this freedom.
- Freedom 2: The freedom to redistribute copies of the program to help others.
- Freedom 3: The freedom to distribute copies of your modified versions. This lets others benefit from your changes.

Git's GPL v2 license grants all four of these freedoms explicitly. Anyone can download, run, study, modify, and redistribute Git. This is not just a legal technicality — it is what has allowed the entire ecosystem around Git to flourish. Tools like GitHub, GitLab, Bitbucket, and hundreds of GUI clients all exist because the underlying Git source code is freely available for others to build on.

3.2 Can a Company Modify and Sell Git Without Sharing Changes?

This is where GPL v2 differs critically from more permissive licenses. Under the GPL, the answer is no — a company cannot take Git, make private modifications, and distribute the modified version as a proprietary product without releasing their changes under the same GPL v2 terms. This is called the copyleft provision, and it is the defining characteristic of the GPL.

The copyleft principle is sometimes described as 'viral' — when GPL-licensed code is incorporated into a larger work that is distributed to users, the entire work must be released under the GPL. This is deliberate. Stallman designed it this way to prevent a scenario where corporations benefit from the community's work while contributing nothing back.

It is worth noting that this applies to distribution, not use. A company can run modified versions of Git internally without ever releasing their changes, because they are not distributing the modified software to others. This distinction matters for understanding why so many companies run heavily customized internal tools built on open-source foundations.

3.3 GPL vs MIT: The Core Philosophical Divide

The difference between the GPL and the MIT license is essentially a difference in philosophy about freedom. The MIT license is maximally permissive: you can do anything you want with MIT-licensed software, including incorporate it into proprietary products without releasing your changes. This is why companies often prefer MIT-licensed dependencies.

The GPL is philosophically stricter. It says: yes, you have freedom — but you cannot use that freedom to take freedom away from others. If you distribute software that includes GPL-licensed code, you must pass those freedoms along to your users.

Which would I choose for something I built? Honestly, it depends on the purpose. If I were building a core utility that I wanted to remain free and prevent corporations from enclosing, I would choose GPL. If I were building a library and I wanted maximum adoption — including by commercial projects that might not release their changes — I would consider MIT. The choice reflects what you value more: maximum freedom-to-use, or maximum freedom-that-propagates.

3.4 Free as in Beer vs Free as in Freedom

The phrase 'free as in free beer' refers to price: software you can get without paying for it. 'Free as in freedom' refers to rights: software you can use, study, modify, and share without restriction. These are completely different concepts, and confusing them leads to misunderstanding the entire opensource landscape.

Git is free in both senses — it costs nothing, and it grants all four essential freedoms. But there are plenty of software programs that are free-of-charge but not free-as-in-freedom. Proprietary freeware, for example, costs nothing but comes with a license that forbids you from modifying or redistributing it. Conversely, you could theoretically sell GPL-licensed software for money — that would be 'nonfree in price' but still 'free as in freedom.' The GPL explicitly permits selling software, as long as you include the source code.

Git's dual freedom is part of why it has become so universal. There was no financial barrier to adopting it, and there was no legal barrier to building commercial services on top of it — as long as those services did not distribute modified versions of Git without sharing the changes.

4. Part A3 — The Ethics of Open Source

4.1 Should All Software Be Open Source?

This question sounds radical, but it deserves a serious answer from both sides.

The argument for universal open source is powerful. Software increasingly controls critical infrastructure — hospitals, power grids, financial systems, voting machines. When the software governing these systems is proprietary, citizens and regulators cannot inspect it for bugs, biases, or backdoors. The opacity of closed-source systems is a form of unaccountable power. Open source makes power transparent and auditable. Given the stakes, there is a real argument that any software touching the public interest should be open.

The argument against is equally serious. Software development is expensive. Companies and individuals invest enormous effort and capital into building software products, and the expectation of being able to sell that work as a proprietary product is part of what drives that investment. If all software had to be open, many commercially viable products would simply not be built, because there would be no economic model to sustain their development. Particularly for niche tools, specialized enterprise software, and consumer applications that require significant design and support work, proprietary models fund ongoing development in ways that community contribution alone cannot always replicate.

My own position is somewhere in the middle. I think infrastructure software — operating systems, compilers, databases, standard libraries — should almost certainly be open source, because the entire ecosystem depends on it and because community ownership prevents any single entity from holding critical tools hostage. But I think there is space for proprietary products that build on open foundations, provided they contribute back where the GPL or similar licenses require them to.

4.2 Is It Ethical to Build a Profitable Business on Free Community Labor?

This question cuts to the heart of modern technology economics. Companies like Red Hat (now part of IBM), Google, Meta, and Microsoft have built enormous portions of their business on top of Linux, Git, Python, and other free software created by volunteers. Is that ethical?

Red Hat's model is often held up as the ethical version: they take Linux, add enterprise support, testing, and certification, and sell that as a product. Critically, they contribute significant engineering resources back to the Linux kernel and other open-source projects. Their business is built on open source, and they actively invest in making it better. This feels like a genuinely fair exchange.

Google and Meta are more complicated. They use Linux at a scale that dwarfs almost any other consumer, and they do contribute code back — both companies have engineers who work full-time on the Linux kernel, and they have released significant open-source projects of their own (Android, TensorFlow, React, etc.). Whether this constitutes 'enough' contribution relative to the value they extract is genuinely debatable.

The concerning scenario is a company that uses GPL-adjacent or MIT-licensed code with no obligation to contribute back, extracts enormous value, and gives nothing to the community. This is legal with permissive licenses. Whether it is ethical is a question the community continues to grapple with, and it has led to interesting new license models — like the Business Source License and Commons Clause — that try to impose contribution expectations that pure open-source licenses do not.

4.3 Standing on the Shoulders of Giants

Every developer working today is building on top of decades of freely shared work. The text editor you use was built with a compiler that someone wrote and shared. The compiler runs on an operating system that thousands of volunteers improved over thirty years. The version control system you use to manage your code was written by Linus Torvalds in ten days because he had access to decades of distributed systems research.

This chain of shared foundations is what the phrase 'standing on the shoulders of giants' means in software. Open source enables rather than undermines original innovation, because it means every new developer starts from a position of enormous accumulated capability rather than rebuilding everything from scratch.

The question of whether open source undermines original innovation is answered by the evidence: some of the most creative and technically impressive software in the world — Linux, Python, Git, Postgres, Firefox — is open source. The collaborative model does not suppress innovation; it redistributes credit and coordinates effort in ways that often produce better results than closed development. What open source does change is who captures the economic value of that innovation, which is a genuinely harder and more contested question.

5. Part B — Linux Footprint: Git on a Linux System

5.1 Installation

Installing Git on a Linux system is remarkably straightforward, which itself reflects the maturity and ubiquity of the project. It is packaged in every major Linux distribution's official repositories.

On Fedora/RHEL/CentOS:

```
sudo dnf install git
```

On Ubuntu/Debian:

```
sudo apt install git
```

The package name is simply 'git' on all major distributions. It can also be compiled from source, which is occasionally done for environments requiring specific features or the very latest version, but for most use cases the packaged version is appropriate and kept current by distribution maintainers.

5.2 Key Directories and Files

Once installed, Git's files live in predictable locations that follow standard Linux filesystem conventions.

1. /usr/bin/git: The main Git executable. This is the command users run directly.
2. /usr/lib/git-core/ : Supplementary Git commands and helper scripts. git-merge, git-fetch, and other subcommands live here as separate executables.
3. /usr/share/doc/git/ : Documentation and copyright information for the installed package.
4. /usr/share/man/man1/ Man: pages for all git commands (git.1, git-clone.1, etc.)
5. ~/.gitconfig: Per-user configuration file. Stores user identity (name, email), preferences, and aliases.
6. .git/ (inside repos): Per-repository directory containing the entire history, objects, refs, and configuration of that repository.

The .git directory is one of the most important design decisions in Git. Unlike centralized systems where history lives on a server, every Git repository carries its complete history locally. This is why Git works offline and why you do not lose everything if a remote server disappears.

5.3 User and Group

Git does not run as a system service or a special user under normal use — it runs as whichever user invokes it. When you run 'git commit' in your terminal, that process runs as your user account with your permissions. This is appropriate because Git is a user-space tool, not a background service.

The exception is when Git is used in server contexts — for example, running git-daemon or serving repositories over SSH. In those scenarios, it is standard practice to create a dedicated 'git' user account with minimal permissions, so that the git server process does not have unnecessary access to the rest of the system. This principle of least privilege is a fundamental Linux security practice.

File ownership matters in Git repositories as well. Repositories created by one user cannot be modified by another without explicit permission changes, which prevents accidental (or malicious) modifications to shared repositories on multi-user systems.

5.4 Service Management

For standard use, Git is not managed as a systemd service — it is invoked directly as needed. But for Git server deployments, the git-daemon service can be enabled through systemd:

```
sudo systemctl enable git-daemon sudo systemctl start git-daemon sudo
systemctl status git-daemon
```

Git installations themselves do not require service management — when you update Git via your package manager, the new version is available immediately upon the next invocation.

5.5 Update Model

This is one of the clearest examples of how the open-source security model works in practice. When a vulnerability is discovered in Git — such as the buffer overflow vulnerabilities found in 2018 and the credential disclosure vulnerability found in 2022 — the process works as follows: the vulnerability is reported, often by security researchers, to the Git security team. The team writes and reviews a patch. The patch is merged into the main Git repository, which is publicly hosted. Distribution maintainers then pull the updated package into their repositories. Users running 'sudo apt upgrade' or 'sudo dnf update' on their Linux systems receive the patched version automatically.

This entire chain — from vulnerability disclosure to patch landing on millions of servers — typically takes days, not months. Because the source code is open, security patches can be reviewed by independent experts before they ship, rather than trusting a vendor to silently fix (or fail to fix) problems in closed code.

6. Part C — The FOSS Ecosystem Around Git

6.1 Dependencies

Git is written primarily in C, with some shell scripts, and it has relatively few external dependencies — a deliberate design choice to make it portable and easy to compile on new platforms. Its core dependencies include:

- zlib — used for compressing and decompressing repository objects. Git stores file snapshots as compressed binary objects.
- OpenSSL or libgcrypt — used for cryptographic operations, including the SHA-1 (and newer SHA-256) hashing that gives each commit a unique fingerprint.
- libcurl — used for HTTP/HTTPS transport when cloning from or pushing to remote repositories over the web.

- Perl — several older Git helper scripts are written in Perl, though newer versions have progressively replaced these with C implementations.
- SSH (OpenSSH) — not a direct dependency of Git itself, but the standard transport layer for authenticated Git operations over SSH.

All of these dependencies are themselves free and open-source software. Git's entire dependency chain is open. This is part of what makes it auditable and trustworthy at the systems level.

6.2 What Git Has Enabled

Git's influence on the software ecosystem cannot be overstated. It did not just replace SVN — it fundamentally changed how software is developed, reviewed, and shipped.

The most visible downstream project is GitHub, founded in 2008 and acquired by Microsoft in 2018 for $7.5 billion. GitHub built a commercial hosting and collaboration platform entirely on top of Git, adding pull request workflows, issue tracking, code review tools, and CI/CD pipelines. The fact that GitHub's core is Git — a free, open-source tool — did not prevent it from becoming one of the most valuable software companies ever acquired.

GitLab followed as an open-source competitor to GitHub, and unlike GitHub, the GitLab software itself is open source (community edition), meaning organizations can self-host their own GitLab instance without paying for the commercial version. This is another example of the open-source model enabling genuine competition.

Beyond hosting platforms, Git enabled the modern DevOps ecosystem. Continuous integration systems like Jenkins, GitHub Actions, GitLab CI, and CircleCI all work by triggering automated builds and tests when changes are pushed to a Git repository. Without Git's distributed design and webhook capabilities, modern software delivery pipelines simply would not exist in their current form.

6.3 Git in the LAMP Stack Context

Git itself is not part of the traditional LAMP (Linux, Apache, MySQL, PHP) stack, but it sits beneath and around every modern web deployment. The code that runs in Apache and MySQL was developed using Git. The tools used to deploy updates to a LAMP stack — Ansible, Puppet, Chef, Capistrano — are all Git-managed and many use Git as a deployment mechanism. The practice of 'git pull on the server to deploy' is so common it has become a standard pattern.

Modern interpretations of the LAMP stack have evolved into MEAN, MERN, and other combinations, but the principle is the same: Git is the connective tissue of all modern software development, even when it is not listed as a stack component.

6.4 Community and Governance

Git is maintained by a relatively small core team with Junio Hamano as the current maintainer — a role he has held since Linus Torvalds handed off maintenance in 2005, just months after creating the project. This long-term stewardship has given Git unusual stability and consistency.

Development happens via the git-mailing list, hosted at vger.kernel.org. Unlike GitHub-centric projects, Git's development is conducted entirely through email patches — which is historically fitting, since Git was designed to support the kernel's email-based development workflow. Patches are proposed, reviewed, and discussed on the list, then applied by Junio.

There are no formal corporate sponsors of Git itself, though many companies (Google, Microsoft, GitHub, GitLab) employ developers who contribute to Git's development as part of their regular work. This creates an interesting dynamic: corporations benefit enormously from Git and contribute engineering to it, but no single corporation controls its direction or governance.

7. Part D — Open Source vs Proprietary Comparison

The table below compares Git against its closest proprietary alternatives: GitHub's proprietary features, Perforce Helix Core (a common enterprise-only VCS), and similar tools.

Dimension	Git (Open Source)	GitHub / Perforce (Proprietary)
Cost	Completely free — no licensing fees for any use case, personal or enterprise.	GitHub.com is free for public repos; enterprise plans cost $21+/user/month. Perforce licenses can cost thousands per seat.
Security (Code Audit)	Source code is fully public. Anyone can inspect, audit, and report vulnerabilities — bugs are fixed faster due to community scrutiny.	Proprietary tools are closed boxes. Security depends entirely on the vendor. Users must trust the company.
Support & Reliability	Community support via mailing lists, Stack Overflow, and documentation. No dedicated support unless you pay a third party.	Enterprise support contracts guarantee SLAs. Vendor-backed reliability is critical for corporations.
Freedom to Modify	Full freedom — you can patch Git, fork it, or build entirely new tools on top of it. Git itself powers GitHub, GitLab, Bitbucket, and more.	No right to inspect or modify. You are entirely dependent on vendor roadmaps and timelines.
Community vs Corporate	Governed by community consensus. Core contributors debate changes publicly. No single company controls Git.	Controlled by Microsoft (GitHub), GitLab Inc., or Perforce. Business priorities can override user needs.
Overall Verdict	Best choice for nearly all use cases — powerful, free, universal, and transparent.	Proprietary hosting platforms add convenience features on top of Git, but the version control engine itself is open source Git.

7.1 Would I Recommend Git for Real-World Deployment?

Unambiguously, yes. I cannot think of a realistic software development scenario where Git would not be the right choice for version control. It is free, it is fast enough for any codebase, it is distributed so it works offline, and it is supported by every development tool, IDE, CI/CD platform, and hosting service on the market. The ecosystem around Git has made it as universal as TCP/IP in networking — not using it would require deliberate effort to avoid it.

The nuanced answer applies not to Git itself but to the hosting layer on top of it. GitHub is convenient and dominant, but it is a proprietary service controlled by Microsoft. Organizations with strong data sovereignty requirements or concerns about vendor lock-in should consider self-hosting GitLab Community Edition, which gives them full control over their repositories while still using Git as the underlying engine. This is exactly the kind of flexibility that the GPL's philosophy enables.

7.2 Would I Contribute to Git?

Contributing to Git is intimidating precisely because it is so fundamental. A bug in Git is a bug that could affect millions of developers. The contribution process — patches submitted via email to the mailing list, reviewed by experienced maintainers — is more formal and demanding than typical GitHub-based open-source contribution. But the learning opportunity is significant. Understanding how Git works internally — object storage, the reflog, the pack protocol — is the kind of deep systems knowledge that makes a developer genuinely better. I would commit to learning the codebase and starting with documentation improvements or small bug fixes before attempting anything in the core data path.

8. Shell Script Documentation

The following five scripts demonstrate practical Linux skills and connect to the open-source concepts explored in this report. Each script is submitted as a separate .sh file in the GitHub repository.

Script 1 — System Identity Report

This script creates a formatted welcome screen displaying system information. It uses variables, command substitution, and echo formatting to present the Linux distribution, kernel version, current user, home directory, system uptime, date/time, and a message about the OS's open-source license (GPL v2 for the Linux kernel).

Concepts used: variables, echo, command substitution with $(), uname, whoami, uptime, date, and basic output formatting with separators.

```
dineshp@LAPTOP-4SSOVD88:~$ nano script1.sh
dineshp@LAPTOP-4SSOVD88:~$ chmod +x script1.sh
dineshp@LAPTOP-4SSOVD88:~$ ./script1.sh
==============================
 Open Source Audit — Dinesh Prajapat
==============================
Software : Git
Kernel   : 6.6.87.2-microsoft-standard-WSL2
User     : dineshp
Uptime   : up 3 hours, 28 minutes
Date     : Mon Mar 30 17:00:25 UTC 2026
```

Script 2 — FOSS Package Inspector

This script checks whether Git (or another named package) is installed on the system, retrieves its version and license metadata from the package manager, and uses a case statement to print a oneline philosophical note about the package. It demonstrates conditional logic for package detection and structured multi-option branching.

Concepts used: if-then-else, case statement, rpm -qi / dpkg -l, pipe with grep.

```
dineshp@LAPTOP-4SSOVD88:~$ nano script2.sh
dineshp@LAPTOP-4SSOVD88:~$ chmod +x script2.sh
dineshp@LAPTOP-4SSOVD88:~$ ./script2.sh
git is installed.
git version 2.43.0
Git: version control system created by Linus Torvalds
```

Script 3 — Disk and Permission Auditor

This script loops through a list of key system directories and reports the permissions, ownership, and disk usage of each. It also checks specifically for Git's configuration directory and reports its permissions separately. This directly relates to the security concepts in Part B of the report.

Concepts used: for loop, arrays, ls -ld, du -sh, awk, cut, conditional directory checking.

```
dineshp@LAPTOP-4SSOVD88:~$ nano script3.sh
dineshp@LAPTOP-4SSOVD88:~$ chmod +x script3.sh
dineshp@LAPTOP-4SSOVD88:~$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 150M
/home => Permissions: drwxr-xr-x root root | Size: 60K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 24K
/etc/git not found
```

Script 4 — Log File Analyzer

This script accepts a log file path and keyword as command-line arguments, reads the file line by line, counts occurrences of the keyword, and prints the last five matching lines. It demonstrates file-based I/O, argument handling, and loop-based processing.

Concepts used: while read loop, if-then, counter variables, $1 and $2 arguments, grep, tail.

```
dineshp@LAPTOP-4SSOVD88:~$ nano script4.sh
dineshp@LAPTOP-4SSOVD88:~$ chmod +x script4.sh
dineshp@LAPTOP-4SSOVD88:~$ ./script4.sh /var/log/syslog
Keyword 'error' found 26 times in /var/log/syslog
Last 5 matching lines:
2026-03-30T13:36:40.272862+00:00 LAPTOP-4SSOVD88 systemd[1]: apport-autoreport.timer - Process error reports when automa
tic reporting is enabled (timer based) was skipped because of an unmet condition check (ConditionPathExists=/var/lib/app
ort/autoreport).
2026-03-30T14:25:08.632487+00:00 LAPTOP-4SSOVD88 kernel: WSL (246) ERROR: CheckConnection: getaddrinfo() failed: -5
2026-03-30T14:55:56.405941+00:00 LAPTOP-4SSOVD88 kernel: WSL (246) ERROR: CheckConnection: connect() failed: 101
2026-03-30T15:00:07.394127+00:00 LAPTOP-4SSOVD88 kernel: WSL (246) ERROR: CheckConnection: connect() failed: 101
2026-03-30T15:47:24.818988+00:00 LAPTOP-4SSOVD88 kernel: WSL (246) ERROR: CheckConnection: getaddrinfo() failed: -5
```

Script 5 — Open Source Manifesto Generator

This creative script asks the user three interactive questions, then generates a personalized opensource philosophy statement using their answers and saves it to a timestamped text file. It demonstrates interactive input, string operations, and file writing.

Concepts used: read for user input, string concatenation, file writing with >, date command, cat.

```
dineshp@LAPTOP-4SSOVD88:~$ nano script5.sh
dineshp@LAPTOP-4SSOVD88:~$ chmod +x script5.sh
dineshp@LAPTOP-4SSOVD88:~$ ./script5.sh
Answer 3 questions:

1. Tool you use daily: Git
2. Freedom means (one word): control
3. What will you build: software

Manifesto saved in manifesto_dineshp.txt

On 30 March 2026, I believe in open source.
I use Git every day.
For me, freedom means control.
I want to build software and share it with the world.
dineshp@LAPTOP-4SSOVD88:~$
```

9. Conclusion

Researching and writing this audit fundamentally changed how I think about the software I use every day. Git is not just a tool — it is a case study in what open source means, what it enables, and why it matters.

Before this project, I used Git the way most people use a calculator: without thinking about what is inside it, who made it, or why they chose to share it. Now I understand that it exists because proprietary software failed someone who needed it, and that person chose to solve the problem publicly. That decision — to build in the open and give others the same freedoms — is why I can use Git for free, why I can inspect its source code, and why hundreds of companies have been able to build entire businesses on top of it.

The GPL v2 license that governs Git is not just a legal document. It is a statement of values. It says: we built this, and we want it to remain free forever, not because we are naive about economics, but because we believe that the freedom to study and modify your tools is a fundamental right for anyone who works with software.

The ethical questions around open source — whether companies exploit community labor, whether all software should be free, what we owe to the tools we stand on — do not have easy answers. But they are the right questions to be asking, and asking them with a specific, real piece of software as the subject makes the answers concrete in ways that abstract philosophy never can.

Every developer today writes code on top of decades of freely shared work. Git is one link in that chain. Understanding it is not optional — it is part of being a professional, and more broadly, part of being a thoughtful participant in the infrastructure of modern life.

References

- GNU Project — The Free Software Definition: gnu.org/philosophy/free-sw.html
- Open Source Initiative — The Open Source Definition: opensource.org/osd
- Eric S. Raymond — The Cathedral and the Bazaar: catb.org/~esr/writings/cathedral-bazaar
- Git Source Repository — kernel.org/pub/software/scm/git/
- SPDX License List — GPL v2 text: spdx.org/licenses/GPL-2.0-only.html
- The Linux Command Line by William Shotts — linuxcommand.org
- Linus Torvalds — Initial Git commit announcement (kernel mailing list, April 2005)
- Git Project Website: git-scm.com